

Compound Protocol Proof

installer

March 3, 2026

Chapter 1

Compound Protocol Consistency

We prove that compound protocols (a global protocol connecting two cluster protocols via translation shims) enforce a consistency property: for any PPO (Preserved Program Order) pair of non-downgrade cache events at the same cache with different addresses, if one event is ordered before the other, then they satisfy a compound linearization order.

1.1 Core Definitions

Definition 1 (Ordered Before). An event e_1 is *ordered before* e_2 if e_1 completes before e_2 starts: $e_1.\text{oEnd} < e_2.\text{oStart}$.

Definition 2 (Encapsulates). An event e_1 *encapsulates* e_2 if e_1 starts before e_2 starts and e_2 ends before e_1 ends: $e_1.\text{oStart} < e_2.\text{oStart} \wedge e_2.\text{oEnd} < e_1.\text{oEnd}$.

Definition 3 (Compound Protocol). A *compound protocol* bundles:

- A global protocol and two cluster protocols,
- Shim axioms for translating between cluster and global protocols,
- A linearization function assigning each request event a linearization event,
- A compound linearization event function,
- Axioms: the global protocol is SWMR, acquire events on a shared-write cannot linearize at cache, and weak write / non-coherent release pairs cannot both linearize at cache.

Definition 4 (Cluster Request Linearization Event). A cluster request linearization event classifies where a cluster cache request linearizes:

- **At cache** (`clusterCacheLin`): the request has permissions and linearizes locally; the linearization event is the cache event itself.
- **At directory or beyond** (`clusterDirLin`): the request does not have permissions locally and must go through the directory (and possibly the global protocol).

Definition 5 (Lazy Compound Linearization Order). A *lazy compound linearization order* between a linearization event e_{lin_1} and a second event e_2 holds if: for every immediate bottom predecessor e_3 of e_2 's directory event at the same address, e_{lin_1} finishes before e_3 's directory event.

Definition 6 (Compound Linearization Order). For a PPO pair (e_1, e_2) , *compound linearization order* holds if either their compound linearization events are ordered (e_{lin_1} is ordered before e_{lin_2}), or the lazy compound linearization order holds between e_{lin_1} and e_2 .

1.2 Auxiliary Structures

Definition 7 (Coherent Write Succeeded by Directory Request). Structure capturing the auxiliary data for a coherent write event that is succeeded by a directory request.

Definition 8 (RCCO-Style Downgrade). Structure capturing the chain of events in an RCCO-style downgrade: a directory event, a downgrade event, and a write-back event.

Definition 9 (Write-Back at Event). Structure recording a VdWriteBack at a given event's entry.

1.3 Contradiction Lemmas

Lemma 10 (Contradiction: Has Perms and No Perms). *A coherent event cannot simultaneously have and lack permissions.*

Lemma 11 (Contradiction: NC Release Has Perms and No Perms). *A non-coherent release request cannot simultaneously have and lack permissions.*

Lemma 12 (Contradiction: Weak Write Has Perms and No Perms). *A weak write request cannot simultaneously have and lack permissions.*

Lemma 13 (Contradiction: Acquire Has Perms and No Perms). *An acquire request cannot simultaneously have and lack permissions.*

Lemma 14 (Contradiction: Coherent Has Perms and No Perms). *A coherent request cannot simultaneously have and lack permissions.*

1.4 Encapsulation Lemmas

Lemma 15 (Directory Lin Encapsulates Global Directory Lin). *If a cluster directory event encapsulates the global directory linearization event, then the encapsulation relation is preserved through the linearization chain.*

Lemma 16 (Event Encaps Dir Lin Encaps Global Dir Lin). *If a cache event encapsulates a cluster directory event, and the cluster directory event encapsulates the global directory linearization event, then the cache event encapsulates the global directory linearization event.*

Lemma 17 (Request Encapsulates Compound Linearization Event). *A request event encapsulates its compound linearization event (generic version for all request types).*

Lemma 18 (NC Release Encapsulates Compound Lin Event). *A non-coherent release request encapsulates its compound linearization event.*

Lemma 19 (Acquire Encapsulates Compound Lin Event). *An acquire request encapsulates its compound linearization event.*

Lemma 20 (Coherent Encapsulates Compound Lin Event). *A coherent request encapsulates its compound linearization event.*

1.5 Linearization Ordering Lemmas

Lemma 21 (Ordered Before and Linearizes at Cache). *If both events linearize at cache and e_1 is ordered before e_2 , then compound linearization order holds. The linearization events are the events themselves.*

Lemma 22 (Cluster Dir Encap Global Lin at Dir). *A cluster directory event encapsulates its compound linearization event when linearizing at directory.*

Lemma 23 (Weak Write/Read and NC Release Linearize at Directory). *If a weak write or read and a non-coherent release both linearize at directory, their linearization events are ordered.*

Lemma 24 (Acquire and Weak Request Linearize at Directory). *If an acquire and a weak request both linearize at directory, their linearization events are ordered.*

Lemma 25 (Weak Write Ordered Before VdWriteBack). *A weak write that is ordered before a VdWriteBack linearizes before the write-back.*

Lemma 26 (State Before Vd of No Between Events). *If there are no between-events on a Vd entry, the state before the Vd event produces a Vd state.*

Lemma 27 (Lazy Coherent Release Ordering (Helper)). *Helper lemma for lazy coherent release ordering.*

Lemma 28 (Lazy Coherent Release Ordering). *Establishes the lazy linearization ordering for coherent releases.*

Lemma 29 (Weak Request to Dir and Coherent Release at Cache). *When a weak request linearizes at directory and a coherent release linearizes at cache, compound linearization order holds.*

Lemma 30 (Contradiction: Weak Write Encapsulating Directory Event). *A contradiction arises if a weak write encapsulates its corresponding directory event.*

1.6 Per-Case Compound Linearization Lemmas

Lemma 31 (CLO: Two Events Encapsulating Their Lin Events). *Handles PPO cases where both events are non-coherent release, acquire, or coherent (e.g. $SC \times SC$, $Rel \times Acq$, $Acq \times Rel$). Shows that each event encapsulates its compound linearization event, so ordered-before is preserved to the linearization events.*

Lemma 32 (CLO: Weak Read and NC Release). *Weak read ordered before a non-coherent release satisfies compound linearization order.*

Lemma 33 (CLO: Weak Write or Read and NC Release). *Generic lemma: weak write or read before a non-coherent release. Handles all sub-cases ($cache \times cache$, $dir \times cache$, $cache \times dir$, $dir \times dir$ linearization).*

Lemma 34 (CLO: Weak Request and NC Release). *Weak non-coherent request ordered before a non-coherent release satisfies compound linearization order.*

Lemma 35 (CLO: Acquire and Weak Request). *Acquire ordered before a weak non-coherent request satisfies compound linearization order.*

Lemma 36 (CLO: Weak Request and Coherent Release). *Weak non-coherent request ordered before a coherent release satisfies compound linearization order.*

1.7 Main Results

Lemma 37 (PPO Cluster Events Satisfy Compound Linearization). *For same-cluster PPO cache events satisfying ordered-before, the compound linearization order holds. The proof is by exhaustive case match on the request types of e_1 and e_2 , dispatching to the appropriate per-case lemma.*

Theorem 38 (Compound Protocol Enforces Consistency). *For any PPO pair of non-downgrade cache events e_1, e_2 at the same cache with different addresses in a behaviour b of a compound protocol: if e_1 is ordered before e_2 , then compound linearization order holds between e_1 and e_2 .*